

CrackMe №1

Task Properties

Name	Value
Author	iseey0u-re
Language	C/C++
Platform	Windows
Difficulty	2.0
Arch	x86-64

Description:

A small crackme where things aren't always what they seem. Look closely at how the logic behaves. Be careful with your debugger!

[Task source](#)

Решения

По референсам [раз](#), [два](#), [три](#) обнаружены техники анти-отладки:

Паттерн NtGlobalFlag + PEB!BeingDebugged

```
mov rax, gs:60h // NtCurrentPeb
cmp byte ptr [rax+0bch], 70h // NtGlobalFlag
jz ...
cmp [rax+2], bl // BeingDebugged
jz ...
```

Встречается по адресам:

1. 140001FCC
2. 140001FF5
3. 1400020DE
4. 140002082

Вызов IsDebuggerPresent с последующей проверкой по адресам:

```
14000205D call cs:IsDebuggerPresent
140002063 mov ebp, eax
...
14000207A test ebx, ebx
14000207C jz ...
```

Анализируем дальше. Цепляемся за валидацию ввода пароля и вывод сообщения о проверке:

```
if ( Size == n15_3 && (!Size || memcmp(Buf1_2, Buf2_4, Size)) )
    Success_ = "Success!";
```

```
else
    Success_ = "Fail!";
```

Кусок кода ниже нас особо не интересует - инструкций типа `goto` да вывода успех\провал не обнаружено. Значит, анализируем что происходит выше.

`n15_3`, как видно из контекста - длина буфера. Значит, длина ввода. Жестких ограничений по вводу не обнаружено - значит, строка произвольной длины (имеется в виду, хардкод-проверки по этой переменной).

Кроме того она используется в сравнении по `memcmp` переменных `Bufl_2` и `Bufl_4` в качестве ограничение по сравниваемым символам. По документации `memcmp` = 0, когда `buffer1` идентичен `buffer2`. В данном случае `!memcmp()` означает 0, то есть совпадение буферов.

`Bufl_2` задается значением `Bufl`

Посмотрим где используется исходный буфер

```
void *Bufl[2];
...
*(_OWORD*)Bufl=0;
...
LOBYTE(Bufl[0])=0;
```

Получается, в этом буфере лежит введенная строка.

`Bufl_4` задается значением `Bufl`, который:

```
memset(____608., ">/<>608.", sizeof(____608.));
...
v17=*( _BYTE*)____608._1 ^ n93
...
sub(Bufl, 1u, v28, v17)
...
```

То есть константа расшифровывается ключом, заданным в переменной `n93` через операцию `xor` и записывается побайтно в `Bufl`.

При этом `n93` изначально имеет значение 93

А при обнаружении отладки через `NtGlobalFlag` или `PEB!BeingDebugged` ключ меняется на 109.

Тогда в первом случае (без отладки) константа происходит

```
>/<>608. ^ 93 = crackmes
```

А во втором (при отладке):

```
>/<>608. ^ 109 = SBQS[]UC
```